

Слайд 1. Титульный — ~15 секунд

Всем привет! Мы команда **«Спокойствие, только спокойствие»**, и наше решение называется **«Спокуха»**.

Мы разработали интеллектуальную систему управления оповещениями, задача которой довольно простая по формулировке: сделать так, чтобы среди тысяч технических событий нужный человек вовремя увидел именно то, на что действительно нужно реагировать.

Слайд 2. Команда — ~15 секунд

Над решением работала команда из шести человек: Максим — капитан, Олег отвечал за архитектуру, Лиза и Анвар — за разработку, Игорь — за инфраструктуру, Алина — за системный анализ.

Поэтому мы постарались посмотреть на решение сразу с нескольких сторон: от бизнес-логики и пользовательского сценария до архитектуры, безопасности и реального прототипа.

Слайд 3. Цель и образ результата — ~25 секунд

Наша главная цель — **снизить потери от простоев бизнес-критичных сервисов и сократить ручную работу с алертами**.

На выходе мы видим единую платформу: у инженера есть личный кабинет с текущими авариями и историей, у администратора — управление пользователями, подписками и маршрутизацией.

Плюс система сама определяет критичность событий, объединяет связанные алерты и доставляет уже обработанное уведомление нужному человеку через TrueConf.

Слайд 4. Задачи — ~20 секунд

Для этого мы должны решить несколько задач.

Во-первых, научиться принимать события сразу из нескольких систем мониторинга и разных форматов.

Во-вторых — автоматически их дедуплицировать, коррелировать, определять критичность и делать короткую понятную сводку.

А со стороны пользователя дать удобный интерфейс, доставку через TrueConf и возможность самостоятельно управлять подписками.

Слайд 5. Почему это вообще нужно — ~40 секунд

Теперь немного про масштаб проблемы.

Сейчас семь систем мониторинга ежедневно генерируют около **5 400 событий**, при этом порядка **65% уведомлений — информационный шум**.

Из-за этого сотрудники первой линии тратят на ручную обработку около **48 человеко-часов в сутки**.

Но самое важное — не сами трудозатраты. Среднее время, в течение которого простой может оставаться незамеченным, составляет около **14 минут**, а один такой инцидент может стоить примерно **3,2 миллиона рублей**.

Поэтому наша логика такая: вместо большого количества разрозненных сообщений создаём единый конвейер — **приняли событие, обработали, поняли важность и доставили его тому, кто действительно должен реагировать**.

Слайд 6. Нефункциональные требования — ~25 секунд

При этом нам важно было сделать не просто красивую AI-демку.

У системы есть конкретные требования: доставка уведомления за **2-3 минуты**, обновление дашборда максимум за 5 минут, работа с пиковыми нагрузками без потери событий.

Критичные алерты получают приоритет, а если основной канал недоступен — предусмотрен fallback и эскалация.

Отдельно мы учитываем безопасность: чувствительные данные не уходят во внешние AI-сервисы, а все действия пользователей и администраторов журналируются.

Слайд 7. Границы системы — ~20 секунд

На этой схеме показано, где находится наша платформа.

Слева приходят события из систем мониторинга — например, Zabbix и Prometheus.

Дальше они проходят через наш коннектор и платформу обработки.

После этого система определяет получателя и через TrueConf отправляет уведомление дежурному инженеру.

Администратор при этом управляет правилами, а Active Directory выступает источником данных о пользователях и их атрибутах.

Слайд 8. Технологии — ~20 секунд

По технологиям мы старались использовать достаточно простой и промышленно применимый стек.

Backend — **Python и FastAPI**, обмен событиями — **Kafka**, хранение — **TimescaleDB**.

Для локального AI используем **vLLM и Qwen**.

Интерфейс работает через nginx, а интеграция с пользователями и уведомлениями строится вокруг Active Directory и TrueConf.

Слайд 9. Как обрабатывается событие — ~35 секунд

Теперь самое интересное — что происходит с самим алертом.

Событие приходит через webhook либо забирается polling-механизмом и приводится к единой схеме.

Дальше работает **детерминированное ядро**. То есть критически важные решения мы не отдаём нейросети.

Система по правилам определяет критичность, удаляет дубли, объединяет связанные сигналы и подавляет так называемые «мигающие» источники.

И важный момент: каждое принятое системой решение сохраняется отдельно. Поэтому всегда можно понять, **почему конкретный алерт был обработан именно так**.

Слайд 10. Маршрутизация, AI и интерфейс — ~40 секунд

После обработки система определяет маршрут: **сервис → ответственная команда → дежурный → канал доставки**.

И только после этого подключается AI.

Это принципиально важная архитектурная идея: **нейросеть помогает, но не управляет критическим контуром**.

Она асинхронно обогащает алерт, формирует более понятное описание и гипотезу первопричины.

Если модель зависла, недоступна или вернула некорректный ответ — ничего страшного. Уведомление всё равно уйдёт, просто будет сформировано детерминированным способом из исходных данных.

То есть AI улучшает качество сообщения, но не становится точкой отказа.

Слайд 11. Что конкретно делает AI — ~35 секунд

Если подробнее, AI у нас выполняет четыре задачи.

Он добавляет семантический контекст, предлагает гипотезу первопричины, определяет приоритет внутри разрешённого словаря и превращает технический алерт в понятный человеку заголовок и текст.

При этом модель работает локально, получает строго ограниченный JSON и не может произвольно менять логику системы.

На тестовом корпусе из **1 053 алертов система сформировала 36 итоговых уведомлений**. То есть объём шума удалось сократить примерно на **96,6%**.

Для нас это один из ключевых результатов прототипа.

Слайд 12. Информационная безопасность — ~30 секунд

Отдельно мы проработали информационную безопасность и разобрали **23 сценария угроз**.

На входе предусматриваем аутентификацию API, TLS, allowlist и rate limiting.

AI работает внутри контура, имеет ограниченный формат ответа и fallback.

Доступ строится на ролевой модели, а изменения правил и подписок журналируются.

При этом мы не говорим, что пилот уже полностью production-ready: остаются риски, связанные, например, с секретами, аудитом, сегментацией и отказом отдельных узлов. Мы их явно фиксируем как задачи перед промышленным внедрением.

Слайд 13. Дерево KPI — ~25 секунд

И всё это должно приводить не просто к техническим метрикам, а к бизнес-эффекту.

Мы связали продуктовые показатели с операционными и финансовыми.

Сокращение информационного шума и ускорение реакции уменьшают трудозатраты сотрудников и длительность простоев.

А уже это напрямую влияет на финансовый результат — то есть технология здесь является не самоцелью, а инструментом сокращения потерь.

Слайд 14. Инфраструктура — ~20 секунд

Для пилота мы также спроектировали инфраструктуру.

Мы разделили её на три блока: приложения, данные и шину событий, а также отдельный AI-контур.

Такое разделение позволяет масштабировать компоненты независимо друг от друга: например, увеличивать количество экземпляров приложения или отдельно добавлять GPU под AI, не перестраивая всю систему.

Слайд 15. Финал — ~15 секунд

В итоге **«Спокуха»** превращает поток из тысяч разрозненных событий в небольшое количество понятных и адресных уведомлений.

Она уменьшает шум, сокращает ручную работу, ускоряет реакцию на действительно критичные инциденты — и при этом не делает AI критической точкой отказа.

Поэтому, как говорил Карлсон: **«Спокойствие, только спокойствие».**

Спасибо!